

Vision-Based Autonomous Navigation of a Differential-Drive Robot via Proximal Policy Optimization in MuJoCo

Ricardo Amiel Acuna Villogas
Data Science
UTEC, Lima, Peru

Salvador Justo Ruben Cordova Flores
Computer Science
UTEC, Lima, Peru

Rodrigo Felix Meza Gavilan
Computer Science
UTEC, Lima, Peru

Abstract—This paper presents the second stage of an autonomous-navigation research project, marking the transition from a discrete value-based baseline to a continuous-control, vision-based policy suitable for physical deployment. The first milestone validated tabular and linearly-approximated Q-Learning in a 5×5 GridWorld. Here we redesign the problem as a continuous-control task in the MuJoCo physics engine, in which a differential-drive robot, referred to as *Roombita*, learns to follow tracks and avoid obstacles from an on-board camera using Proximal Policy Optimization (PPO). The architecture couples a Gymnasium environment, a differential-drive command interface, a dense geometry-based reward, and a two-stage perception pipeline in which the policy is trained on clean simulator segmentation and deployed through a learned U-Net segmenter (mean IoU 0.881), decoupling control from perception for sim-to-real transfer. We train an optimized PPO agent for 10^6 steps using generalized State-Dependent Exploration (gSDE), frame stacking, learning-rate decay, a control-smoothness penalty, and eight parallel environments spanning a curriculum of six closed tracks. A systematic ablation over training checkpoints, perception modality, and track geometry shows that (i) the learned segmenter reproduces the performance of privileged simulator masks almost exactly (goal rate 0.20 in both cases), confirming that perception is *not* the bottleneck; (ii) task competence emerges only after 700k steps and peaks at 900k (0.30 goal rate) before regressing at 10^6 steps, empirically motivating best-checkpoint selection; and (iii) the dominant failure mode across all geometries is vehicle roll-over (flip rate 0.70–1.00), a physical-dynamics limitation rather than a perceptual one. We analyze these results and outline the domain-randomization and dynamics-regularization steps required to close the remaining sim-to-real gap.

Index Terms—Reinforcement Learning, PPO, MuJoCo, Gymnasium, autonomous navigation, mobile robotics, continuous control, semantic segmentation, sim-to-real, Roombita.

I. INTRODUCTION

Autonomous navigation is a central challenge in mobile robotics: an agent must move through an environment while respecting physical constraints, avoiding obstacles, and reaching a target efficiently. In the first milestone of this project, the navigation problem was formulated as a Markov Decision Process (MDP) in a 5×5 GridWorld maze and solved with tabular and linearly-approximated Q-Learning. That stage validated the reward design, the transition structure, and the basic RL loop under a simplified discrete setting.

However, a real mobile robot does not act by selecting symbolic moves such as *North* or *East*. It controls actuators continuously, processes noisy visual observations, and reacts to dynamic geometry and contact forces. For that reason, this second stage migrates the project from a discrete maze to a continuous-control MuJoCo simulation. The new environment models a small differential-drive robot that follows a track in a cluttered laboratory scene with probabilistic obstacles, an on-board camera, and configurable trajectories. Value-based tabular methods cannot represent this problem: the action space is continuous and the observation is a high-dimensional image. We therefore adopt PPO [1], an on-policy actor-critic method that directly optimizes a stochastic policy over continuous actions while constraining each update to preserve stability.

The contributions of this stage are the following:

- A Gymnasium-compatible MuJoCo environment for a differential-drive robot with a physically meaningful *Twist* command interface, a dense geometry-based reward defined by arc-length projection, and four observation modes ranging from privileged state to raw pixels.
- A two-stage perception design that trains the control policy on clean simulator segmentation and deploys it through an independently-trained U-Net segmenter, decoupling control from perception so that the same policy can consume either simulated or real imagery.
- An *optimized* PPO configuration for continuous robotic control — generalized State-Dependent Exploration, frame stacking, linear learning-rate decay, a control-smoothness penalty, and multi-track parallel rollouts — trained for 10^6 steps.
- A systematic quantitative and qualitative ablation over training progress, perception modality, and track geometry that isolates the perception gap from the physical-stability gap and yields concrete guidance for sim-to-real transfer.

II. RELATED WORK

Reinforcement Learning has been widely studied for autonomous decision-making when an agent must learn a policy from interaction rather than from a hand-crafted model [2], [3]. Early value-based methods such as Q-Learning and its

deep variant DQN solved spatial navigation in discrete mazes without a pre-existing environment model [4]. These methods, however, require discretizing the action space, which is inefficient for the smooth wheel commands of a physical robot.

For continuous control, policy-gradient methods are preferred. PPO constrains the policy update through a clipped surrogate objective, achieving a favorable balance between sample efficiency and stability [1], and is typically combined with Generalized Advantage Estimation for low-variance advantage targets [5]. Exploration in continuous action spaces benefits from state-dependent, temporally-coherent noise; generalized State-Dependent Exploration (gSDE) produces smoother trajectories than independent Gaussian noise and is specifically designed for robotic learning [6]. We adopt both techniques through the Stable-Baselines3 implementation [7].

Physics simulators such as MuJoCo provide fast, contact-rich rigid-body dynamics and are a standard substrate for robotic RL [8]; we expose the simulation through the Gymnasium API [9]. The central obstacle to deploying simulation-trained policies is the *sim-to-real* gap [10]. Two complementary strategies address it: domain randomization, which varies simulator parameters so that reality appears as one more variation [11]; and perceptual abstraction, which trains control on a compact, transferable representation (e.g., semantic segmentation) produced by a learned network [12], [13]. Our design combines both: we randomize obstacles, lighting, and track geometry during training, and we insert a U-Net segmenter between the camera and the policy so that control never observes raw pixels directly.

III. FROM GRIDWORLD Q-LEARNING TO ROOMBITA PPO

The project evolved from a discrete navigation problem into a continuous robotic control task. Table I summarizes the main differences between the previous milestone and the current implementation.

TABLE I
EVOLUTION OF THE PROJECT ACROSS MILESTONES

Aspect	Milestone 1	Current Stage
Environment	5×5 discrete Grid-World maze	MuJoCo laboratory simulation
Agent	Rover in a maze	Differential-drive Roombita robot
Algorithm	Tabular / VFA Q-Learning	PPO (actor-critic) with gSDE
Actions	Discrete: N, S, E, W	Continuous <i>Twist</i> (v, ω)
Observations	Cell coordinates / features	State vector, segmentation mask, RGB, or learned segmentation
Perception	None	U-Net segmenter (mIoU 0.881)
Obstacles	Fixed or randomized cells	Probabilistic obstacle slots
Objective	Reach a goal cell	Follow a track, avoid obstacles, complete the loop

The first milestone demonstrated that a dense reward could produce goal-directed behavior in a stationary MDP. The

current stage requires a method that controls wheel velocities directly from images. PPO satisfies this requirement because the policy maps observations to continuous motor commands, and because its clipped objective tolerates the noisy, contact-rich dynamics of the simulator.

IV. SYSTEM OVERVIEW

The implementation is organized as a modular RL pipeline. The simulator builds the arena and track, the Gymnasium environment exposes observations and actions, PPO trains a policy, and the evaluation scripts test the saved policy either on simulator masks or through the learned segmenter. Fig. 1 summarizes this workflow.

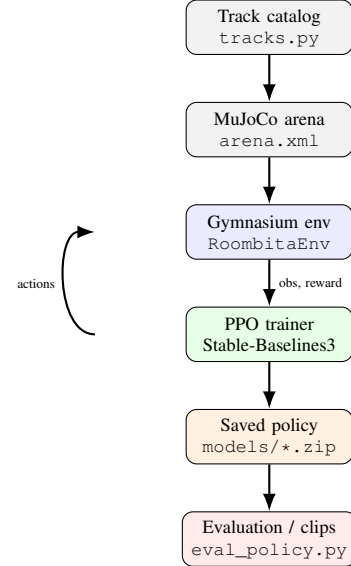


Fig. 1. Architecture of the Roombita RL pipeline. The environment is generated in MuJoCo, wrapped as a Gymnasium task, and trained with PPO; evaluation may substitute the learned segmenter for the simulator’s ground-truth mask.

V. MDP FORMULATION

The Roombita navigation task is an MDP $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, where $\mathcal{S}$ is the observation space, $\mathcal{A}$ the continuous action space, P the transition dynamics induced by MuJoCo, R the reward, and γ the discount factor. Unlike the GridWorld formulation, P is no longer a deterministic move between cells: the next state depends on wheel-velocity commands, orientation, contact dynamics, obstacles, and the robot’s physical interaction with the arena.

A. Action Space: a *Twist* Command Interface

A key design decision, and a correction with respect to the milestone-1 report, is that the policy does *not* emit raw wheel velocities. The action is a normalized two-dimensional *Twist*:

$$a_t = [v, \omega], \quad v, \omega \in [-1, 1], \quad (1)$$

where v maps to a linear velocity in $[-v_{\max}, v_{\max}]$ ($v_{\max} = 0.6$ m/s) and ω to an angular velocity in $[-\omega_{\max}, \omega_{\max}]$ ($\omega_{\max} = 3.0$ rad/s). Inside the environment, the *Twist* is

converted to left/right wheel angular velocities through the differential-drive kinematics

$$\omega_L = \frac{v - \omega \ell}{r}, \quad \omega_R = \frac{v + \omega \ell}{r}, \quad (2)$$

with wheel radius $r = 0.028$ m and half-axle $\ell = 0.085$ m. Emitting a Twist, rather than wheel commands, means the policy produces exactly the control signal a real differential-drive robot receives, which is the natural interface for sim-to-real deployment. Table II lists representative commands.

TABLE II
CONTINUOUS TWIST-ACTION INTERPRETATION

v	ω	Expected Motion
+1.0	0.0	Move straight forward
-1.0	0.0	Move straight backward
0.0	+1.0	Rotate in place (left)
0.0	-1.0	Rotate in place (right)
+0.7	+0.7	Forward-left curve
+0.7	-0.7	Forward-right curve

B. Observation Space

The environment supports four observation modes, making the framework scalable from low-dimensional control to vision-based control (Table III). The `state` mode exposes privileged geometry and is used only as a fast, idealized baseline; it is *not* sim-to-real, because a physical robot cannot measure its exact pose on the track. The `mask`, `rgb`, and `seg` modes depend only on the on-board camera and constitute the realistic perception path. In this work the policy is trained and evaluated on the visual path. For visual modes, four consecutive frames are stacked along the channel dimension, providing the network with motion cues (velocity, turning rate) that a single frame lacks; after the standard channels-first transposition this yields a $12 \times 84 \times 84$ tensor consumed by a convolutional policy.

VI. REFINED REWARD FUNCTION

The reward follows a dense shaping strategy driven by *signed arc-length progress*: the robot’s pose is projected onto the track polyline to obtain the progress increment Δs , the signed lateral error, and the local tangent. The per-step reward is a weighted sum of progress, safety, obstacle-interaction, and terminal-event terms,

$$R_t = w_p \Delta s + r_{\text{safe}} + r_{\text{avoid}} + r_{\text{ret}} - r_{\text{col}} - r_{\text{off}} - r_{\text{stuck}} - r_{\text{smooth}}, \quad (3)$$

augmented by terminal rewards for arrival, wrong-direction arrival, timeout, and flip. Table IV lists the tuned weights used in this stage.

Two refinements over the milestone baseline proved important. First, the *return-to-track* bonus is gated on real forward progress: it is granted only if the robot advanced more than 0.15 m along the track between leaving and re-entering it. Without this gate, an agent can oscillate across the track edge

to farm the bonus, a reward-hacking behavior we observed and eliminated. Second, a *control-smoothness* penalty

$$r_{\text{smooth}} = \kappa \left(\frac{|\omega|}{\omega_{\text{max}}} + \|a_t - a_{t-1}\| \right), \quad \kappa = 0.02, \quad (4)$$

discourages aggressive turning and abrupt command changes, which reduces jitter and, importantly, the tendency of the light chassis to tip over.

VII. LEARNED VISUAL PERCEPTION FOR SIM-TO-REAL

To decouple control from raw pixels, we train a lightweight encoder-decoder segmenter (a MiniUNet, following the U-Net design [12]) that maps an RGB frame to a per-pixel class map (background, road/line, obstacle). The network is trained by supervised learning on clips recorded in simulation, using the engine’s ground-truth segmentation as labels; it reaches a mean IoU of **0.881** on a held-out split after 100 epochs. Crucially, the class palette of the predicted mask is identical to the simulator’s mask palette, so the distribution of observations at deployment matches the distribution the policy saw during training.

This yields the two-stage scheme central to the paper: PPO is trained on *clean* simulator masks (`mask` mode) for speed and stability, while at evaluation the frozen segmenter is inserted so that the pipeline becomes $\text{RGB} \rightarrow \text{U-Net} \rightarrow \text{mask} \rightarrow \text{PPO}$ (`seg` mode). Because the segmenter accepts any RGB image, the identical policy can, in principle, be driven from a real camera without retraining the controller — reducing the sim-to-real problem to the perception module alone. Section X shows that this substitution incurs almost no performance loss.

VIII. OPTIMIZED PPO METHODOLOGY

PPO is an on-policy actor-critic algorithm that learns a policy $\pi_\theta(a|s)$ and a value function $V_\phi(s)$ while preventing destructive updates through a clipped surrogate objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (5)$$

$$\rho_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}, \quad (6)$$

where the advantage $\hat{A}_t$ is estimated with GAE [5] and ϵ is the clip range. For image modes the feature extractor is the NatureCNN convolutional encoder, followed by separate 256×256 policy and value heads. Exploration uses gSDE [6], which draws state-dependent, temporally-correlated noise and produces smoother control than independent Gaussian sampling — a property well suited to a physical drivetrain.

Table V lists the optimized configuration and the rationale for each choice. Relative to the milestone-1 defaults, the main changes are: eight parallel environments spanning a six-track curriculum; a rollout horizon of 16,384 steps; a linearly decayed learning rate; gSDE; four-frame stacking; and the control-smoothness penalty. Algorithm 1 outlines the training loop.

TABLE III
OBSERVATION MODES IMPLEMENTED IN THE ROOMBITA ENVIRONMENT

Mode	Observation	Description	Policy
state	10-dimensional vector	Lateral error, heading error (sine/cosine), linear and angular velocity, on-track flag, remaining arc-length, obstacle distance and bearing. <i>Privileged</i> .	MlpPolicy
mask	$84 \times 84 \times 3$ segmentation	Ground-truth simulator mask separating track, line, obstacles, and background.	CnnPolicy
rgb	$84 \times 84 \times 3$ RGB image	Raw camera view including floor, clutter, people, and lighting variation.	CnnPolicy
seg	RGB $\rightarrow$ U-Net $\rightarrow$ mask	RGB is segmented by the learned network before the policy receives it. <i>Deployment path</i> .	CnnPolicy

TABLE IV
TUNED REWARD COMPONENTS (WEIGHTS USED IN THIS STAGE)

Component	Purpose	Weight
Progress	Signed forward progress on track	+8.0/m
Safe step	Directional safe movement	± 0.02
Arrival	Completing the track	+50.0
Speed bonus	Faster completion	up to +15.0
Wrong direction	Reaching the goal backward	-30.0
Timeout (base)	Not finishing in time	-5.0
Remaining distance	Unfinished arc-length	-1.5/m
Collision	Obstacle contact (event, 1 s cooldown)	-8.0
Avoidance	Passing an obstacle safely	+3.0
Off-track	Leaving the track	-0.20/step
Return (gated)	Returning after real progress	+0.8
Stuck	Stagnation against an object	-1.5/s
Flip (terminal)	Roll-over / instability	-15.0
Smoothness	Aggressive/jerky commands	$-\kappa(\cdot)$

TABLE V
OPTIMIZED PPO TRAINING CONFIGURATION

Parameter	Value	Rationale
Total timesteps	10^6	Budget for visual control
Parallel envs	8	Throughput; decorrelated data
Curriculum	6 tracks	Geometry generalization
Rollout horizon	16,384	2048 steps $\times$ 8 envs
Batch size	256	Stable minibatch updates
Epochs / update	10	Standard PPO reuse
Learning rate	2.5×10^{-4} , linear $\rightarrow$ 0	Late-stage stability
Clip range ϵ	0.2	Trust-region size
Discount γ	0.995	Long effective horizon
GAE λ	0.95	Bias/variance trade-off
Entropy coef.	0.005	Sustained exploration
Exploration	gSDE ($f_s=4$)	Smooth continuous control
Frame stack	4	Motion cues for CNN
Smoothness κ	0.02	Anti-jitter / anti-flip
Camera / obs	84×84 , mask	Trained on clean masks
Obstacles	6 slots, $p=0.8$	Dense clutter
Time limit	20 s	Tight episode budget

Algorithm 1 Optimized PPO Training Loop for Roombita

- 1: Build 8 parallel MuJoCo envs (six-track curriculum, frame stacking)
- 2: Initialize CNN actor-critic with gSDE exploration
- 3: **for** each training iteration **do**
- 4: Collect 16,384-step rollouts with the current policy
- 5: Compute returns and GAE advantages
- 6: Optimize the clipped objective for 10 epochs (batch 256)
- 7: Decay the learning rate linearly with progress
- 8: Log reward, length, KL, clip fraction, value loss, action std
- 9: Checkpoint every 10^5 steps for best-model selection
- 10: **end for**
- 11: Save the final policy and training curves

IX. EXPERIMENTAL SETUP

The agent was trained with the configuration of Table V on a curriculum of six closed tracks (oval, S-curve, zigzag, simple line, hairpin, complex), one per parallel environment, with per-episode randomization of obstacles and brightness. Checkpoints were saved every 10^5 steps. The full implementation is publicly available [14].

Evaluation follows a fixed protocol distinct from training, to test generalization: obstacle slots = 4, obstacle probability = 0.6, time limit = 45 s, deterministic actions, and 10 episodes per condition with fixed seeds. For *seg* evaluation the RGB frame is rendered at 256×256 , segmented by the frozen U-Net, and resized to 84×84 before entering the policy. We report the mean episode reward, mean episode length, and the empirical distribution of episode outcomes (goal, flip, timeout).

X. RESULTS AND ANALYSIS

A. Training Dynamics

Fig. 2 shows the measured training curves over the 10^6 -step run. The mean episode reward (panel a) increases monotonically from -19.9 to -12.6 and plateaus after roughly 800k steps. It is important to read this value in context: it is the reward *averaged across all six tracks*, including the hardest ones (hairpin, complex, zigzag) under dense obstacles and a tight 20 s budget; the negative mean therefore reflects the

difficulty of the averaged curriculum, not a failure to learn — per-track evaluation (Section X-B) reveals clearly positive performance on the easier geometries. Every secondary indicator confirms healthy, stable convergence: the value function’s explained variance (panel c) rises from ≈ 0 to 0.71; the approximate KL divergence (panel d) collapses from an initial 9.2 — the transient instability typical of early gSDE rollouts — to 3×10^{-3} ; the clip fraction (panel e) falls from 0.99 to 0.01, indicating that updates eventually remain within the trust region; and the policy’s action standard deviation (panel f) contracts from 1.01 to 0.47 as exploration gives way to exploitation, consistent with the decaying learning rate. The mean episode length (panel b) grows from 114 to 177 steps, showing that the agent learns to survive and drive farther before terminating.

B. Quantitative Ablation

To characterize the policy beyond the aggregate training reward, we ran three ablations (Fig. 3; numbers in Table VI).

(a) Learning progress. Evaluating the checkpoints on the oval in *seg* mode reveals a clear behavioral progression. At 100k steps the agent is passive and drives too slowly, timing out in every episode with a strongly negative reward (−187.9). Between 300k and 500k it becomes mobile but unstable, flipping in 100% of episodes. Task competence emerges only after 700k steps (0.10 goal rate) and *peaks at 900k with a 0.30 goal rate and mean reward 47.0*, before regressing slightly at 10^6 steps (0.20 goal rate). This non-monotonic late-stage behavior is a direct, empirical argument for selecting the best checkpoint by periodic evaluation rather than shipping the last one — exactly the mechanism our training pipeline provides.

(b) Perception gap. Replacing the privileged simulator mask (*mask*) with the learned segmenter (*seg*) leaves the goal rate unchanged (0.20 in both cases) and lowers the mean reward only modestly (37.9 $\rightarrow$ 30.1). The U-Net perception module therefore reproduces the behavior obtained under ground-truth perception almost exactly, confirming that perception is *not* the performance bottleneck and validating the two-stage sim-to-real design.

(c) Track generalization. On unseen or harder geometries the agent retains partial competence: in-distribution oval and S-curve reach a 0.20 goal rate, whereas the out-of-distribution straight track (absent from the training curriculum) and the sharp zigzag drop to 0.10 and 0.00. Across every condition, however, roll-over dominates the failures, with flip rates between 0.70 and 1.00 and large reward variance (std 50–70), the signature of a system operating near a stability threshold.

C. Qualitative Behavior

Fig. 4 illustrates the perception pipeline and two representative rollouts. The top row shows, for a single frame, the RGB camera image, the ground-truth simulator segmentation, the U-Net prediction, and the obstacle overlay; the close agreement between the second and third panels is the visual counterpart of the negligible perception gap measured above. The middle row traces a successful (goal) episode in which the robot tracks

TABLE VI
QUANTITATIVE ABLATION (10 EPISODES/CONDITION, DETERMINISTIC POLICY)

Condition	Setting	Reward	Goal	Flip
Checkpoint (oval, seg)	100k	−187.9	0.00	0.00
	300k	−6.8	0.00	1.00
	500k	−8.5	0.00	1.00
	700k	+18.5	0.10	0.90
	900k	+47.0	0.30	0.70
Perception (oval, 10^6)	<i>mask</i> (oracle)	+37.9	0.20	0.80
	<i>seg</i> (learned)	+30.1	0.20	0.80
Track gen. (seg, 10^6)	straight (OOD)	−17.8	0.10	0.90
	oval	+18.7	0.20	0.80
	s_curve	+3.8	0.20	0.80
	zigzag	+1.6	0.00	1.00

the line and completes the loop; the bottom row traces a failure episode in which an aggressive turn near the track boundary induces a roll-over, the characteristic terminal event of this agent.

XI. DISCUSSION: STABILITY AND THE SIM-TO-REAL GAP

The ablations separate two gaps that are often conflated. The *perception gap* — the loss from replacing privileged simulator segmentation with a network that sees only RGB — is essentially closed here (mIoU 0.881; identical goal rate under *mask* and *seg*). What remains is a *physical-dynamics gap*: the agent’s dominant failure is roll-over, not perceptual error.

Several factors explain the high flip rate. The chassis is a light ellipsoidal shell (0.9 kg) on two driven wheels and two passive casters, and the action space permits angular velocities up to 3.0 rad/s; a sharp turn, particularly during an obstacle contact, can raise the center of mass past the 60° tilt threshold that the environment treats as a terminal flip. The large episode-reward variance (std 50–70) and the run-to-run differences we observed under nominally identical seeds are consistent with a chaotic, contact-rich system operating near this stability boundary, where a negligible perturbation in the predicted mask or a floating-point difference in a GPU inference is enough to tip the outcome from goal to flip. The control-smoothness penalty and gSDE both mitigate but do not eliminate this behavior.

These observations translate directly into design guidance. Reducing $\omega_{\max}$, lowering the center of mass, widening the wheelbase, or adding an explicit angular-acceleration limit would attack the flip mechanism at its source. On the learning side, the non-monotonic late-stage reward argues for best-checkpoint selection, and the weak transfer to the out-of-distribution straight track argues for a broader curriculum. Finally, because the perception module already generalizes across the simulator’s visual variation, the highest-value next step for true sim-to-real transfer is to harden the segmenter against real-camera statistics (noise, blur, lighting) rather than to further tune the controller.

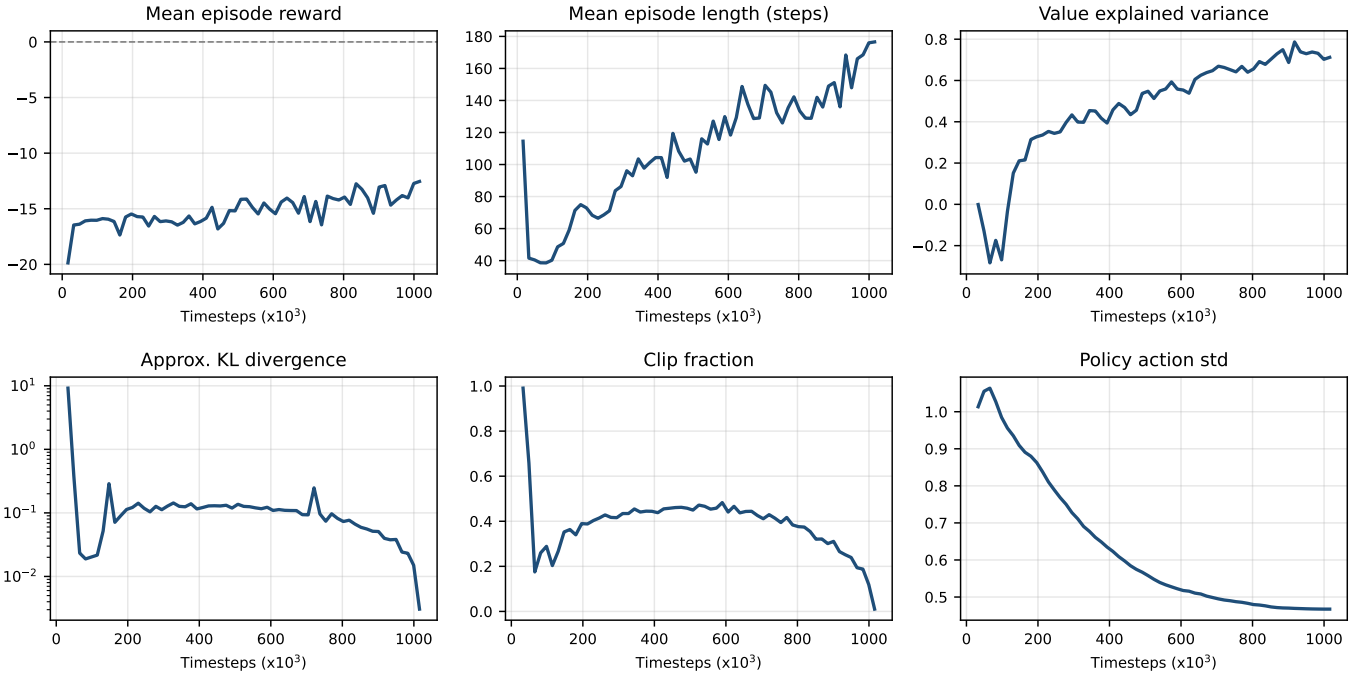


Fig. 2. Measured PPO training dynamics over 10^6 steps (real logs, not a placeholder). (a) Mean episode reward improves monotonically yet remains negative, reflecting the averaged six-track curriculum rather than a learning failure. (b) Mean episode length grows as the agent survives longer. (c) Value explained variance rises to 0.71, evidencing an accurate critic. (d) Approximate KL divergence (log scale) decays from an early gSDE transient to 3×10^{-3} . (e) Clip fraction settles near 0.01, confirming trust-region behavior. (f) Action standard deviation contracts from 1.01 to 0.47, marking the exploration-to-exploitation transition under linear learning-rate decay.

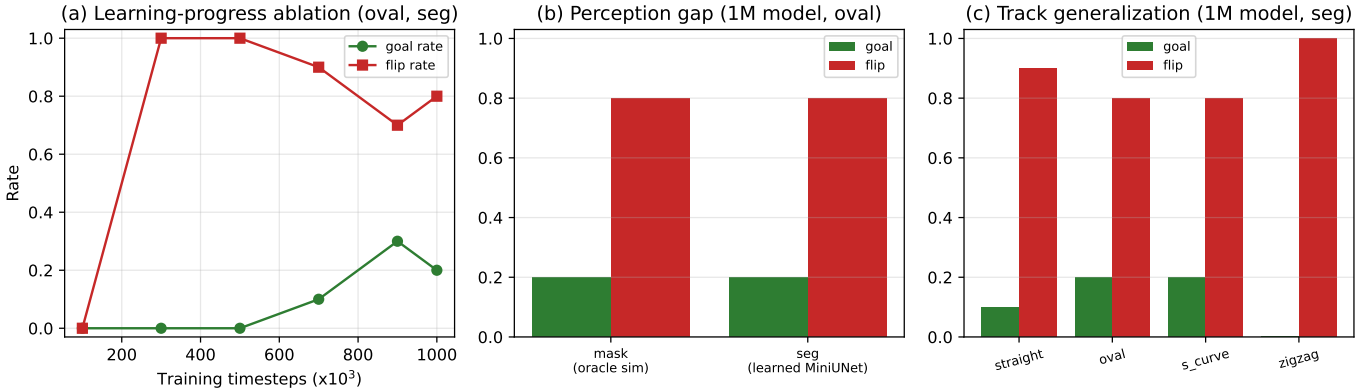


Fig. 3. Quantitative ablation of the trained policy. (a) Goal and flip rates versus training steps on the oval (*seg*): competence emerges after 700k and peaks at 900k. (b) Perception gap on the oval at 10^6 steps: the learned segmenter (*seg*) matches the privileged mask (*mask*) in goal rate. (c) Generalization across track geometries: in-distribution tracks outperform the out-of-distribution straight track and the sharp zigzag, with roll-over dominating all failures.

XII. LIMITATIONS AND FUTURE WORK

Three limitations frame the next stage. First, the physical-stability gap: roll-over remains the dominant failure and must be addressed through chassis geometry and action-space constraints before hardware trials. Second, the budget: 10^6 steps is modest for pixel-based control, and the peak at 900k suggests that a longer schedule with best-model selection and mild regularization could raise the goal rate. Third, sim-to-real fidelity: our evaluation still uses simulator-rendered RGB, so the reported perception gap is the oracle-versus-learned gap within

simulation; quantifying transfer to a real camera requires collecting real imagery and applying domain randomization [11] to the segmenter. Future work will (i) regularize the dynamics and re-evaluate flip rates, (ii) extend the curriculum and training horizon, and (iii) deploy the frozen policy on a physical differential-drive platform through the Twist interface.

XIII. CONCLUSION

This paper documented the transition of an autonomous-navigation project from a discrete Q-Learning baseline to a continuous-control, vision-based PPO agent in MuJoCo. The

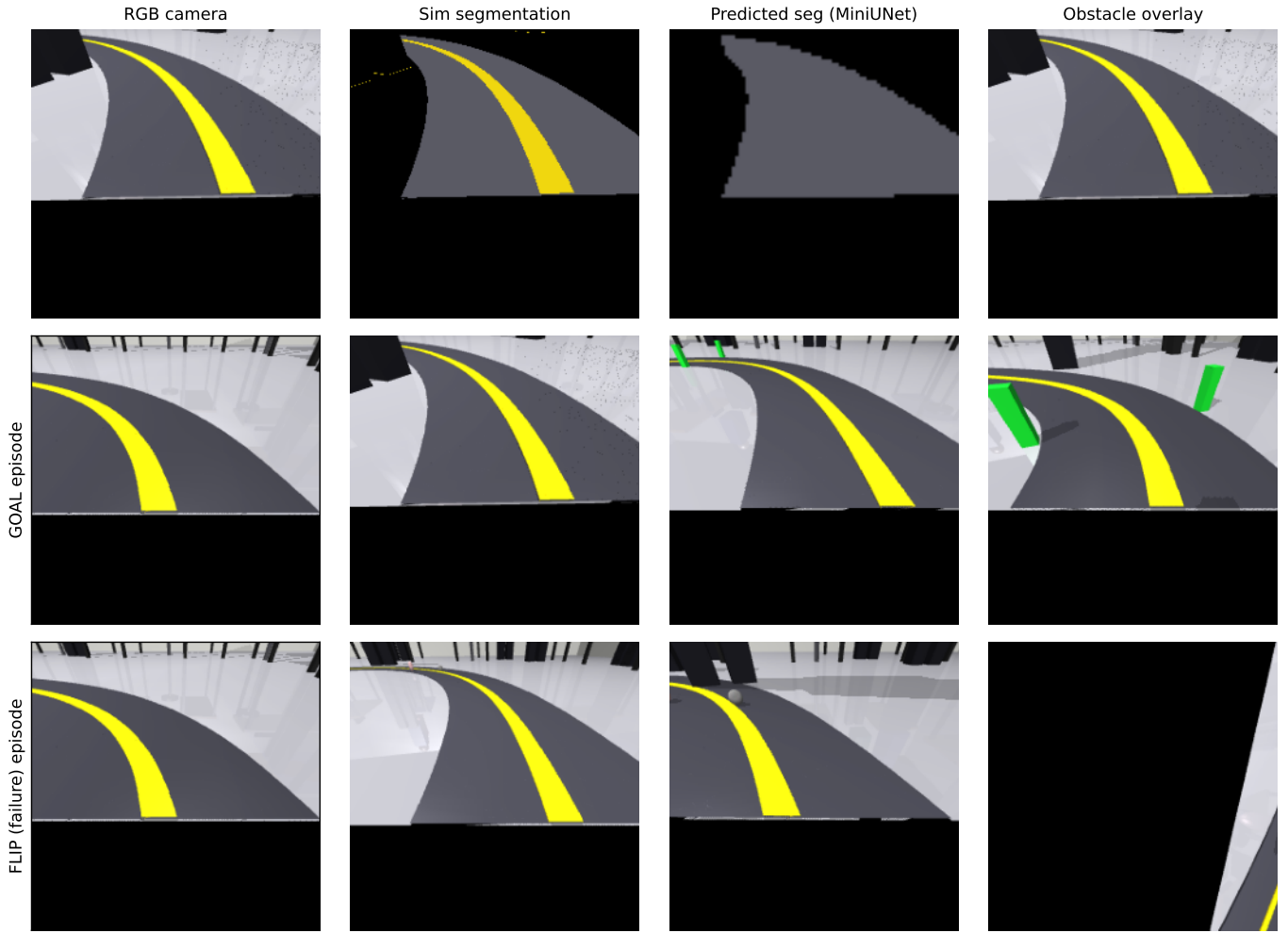


Fig. 4. Qualitative evaluation on the oval (seg mode, 10^6 -step policy). **Top:** perception pipeline for one frame — RGB, ground-truth mask, U-Net prediction, and obstacle overlay; predicted and ground-truth masks are nearly indistinguishable. **Middle:** a successful goal episode. **Bottom:** a failure episode ending in roll-over, the dominant terminal event.

framework contributes a Twist command interface, a dense arc-length reward with an anti-hacking return gate and a smoothness penalty, and a two-stage perception design that trains control on clean masks and deploys it through a learned U-Net segmenter. Training an optimized PPO configuration for 10^6 steps produced a policy whose learning dynamics are stable and whose perception module reproduces privileged-mask performance almost exactly. A systematic ablation isolated the remaining obstacle: not perception, but physical roll-over stability, with task competence emerging after 700k steps and peaking at 900k. These results turn a single training run into concrete, measured guidance — dynamics regularization, best-checkpoint selection, and segmenter domain randomization — for closing the sim-to-real gap in the next stage.

APPENDIX A PROJECT REPOSITORY

The complete implementation of the proposed autonomous navigation framework is publicly available in the following

GitHub repository:

<https://github.com/RodriFMG/RobotinRL>

The repository contains:

- MuJoCo simulation environment and robot model.
- PPO training pipeline and evaluation scripts.
- Reward function implementation.
- U-Net semantic segmentation model and training code.
- Experiment configurations and reproducibility scripts.
- Documentation describing installation, execution, and project structure.

This repository accompanies the manuscript and provides all resources required to reproduce the experiments presented in this paper.

REFERENCES

- [1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.

- [2] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018. [Online]. Available: <http://incompleteideas.net/book/the-book-2nd.html>
- [3] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *The International Journal of Robotics Research*, vol. 32, no. 11, pp. 1238–1274, 2013.
- [4] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [5] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2016.
- [6] A. Raffin, J. Kober, and F. Stulp, "Smooth exploration for robotic reinforcement learning," in *Proc. 5th Conf. Robot Learning (CoRL)*, ser. Proceedings of Machine Learning Research (PMLR), vol. 164, 2022, pp. 1634–1644. [Online]. Available: <https://proceedings.mlr.press/v164/raffin22a.html>
- [7] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: <http://jmlr.org/papers/v22/20-1364.html>
- [8] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033.
- [9] M. Towers, A. Kwiatkowski, J. K. Terry, J. U. Balis, G. De Cola, T. Deleu, M. Goulão, A. Kallinteris, M. Krimmel, A. KG *et al.*, "Gymnasium: A standard interface for reinforcement learning environments," *arXiv preprint arXiv:2407.17032*, 2024.
- [10] W. Zhao, J. Peña Queralta, and T. Westerlund, "Sim-to-real transfer in deep reinforcement learning for robotics: a survey," in *Proc. IEEE Symp. Series on Computational Intelligence (SSCI)*, 2020, pp. 737–744.
- [11] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2017, pp. 23–30.
- [12] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, ser. Lecture Notes in Computer Science, vol. 9351. Springer, 2015, pp. 234–241.
- [13] L. Tai, G. Paolo, and M. Liu, "Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2017, pp. 31–36.
- [14] R. F. Meza Gavilan, "RobotinRL: Roombita RL arena," GitHub repository, 2026. [Online]. Available: <https://github.com/RodriFMG/RobotinRL>